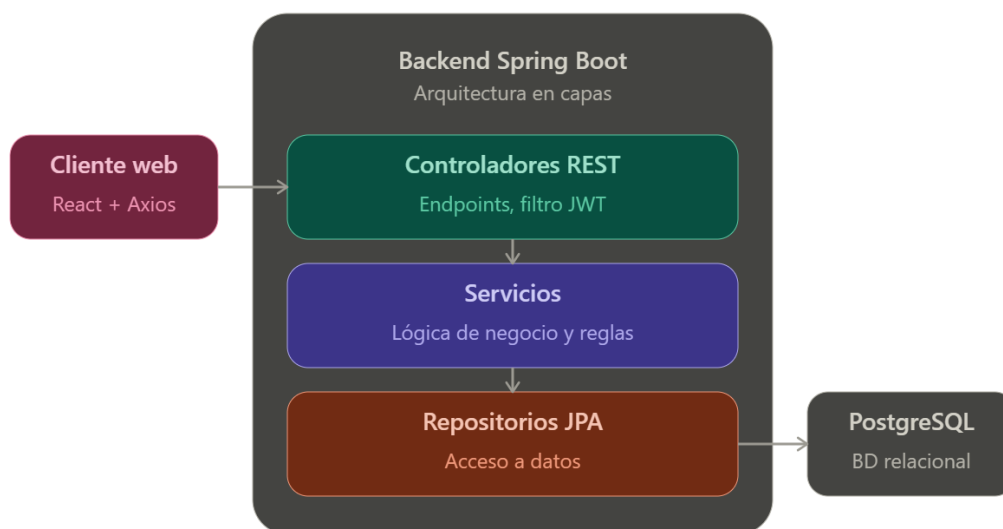


Con Spring Boot + grupo de 2 + arrancando desde cero, hay que ser honestos primero: les quedan más o menos 36-40 horas reales (incluyendo dormir) hasta el domingo al mediodía, para un sistema con 9 módulos, auth con roles, API documentada, tests, despliegue y video. Es alcanzable, pero solo si priorizan por puntaje de la rúbrica y trabajan en paralelo desde ya, sin intentar hacer todo "perfecto" en cada módulo.

Les armo el plan completo: arquitectura, estructura de carpetas, modelo de datos, división de trabajo entre los dos, y cronograma por bloques de tiempo.



Con la arquitectura ya clara (cliente React, tres capas en el backend, Postgres), vamos al resto del plan.

Modelo de datos (entidades principales)

No hace falta un diagrama ER separado; con esta lista alcanza para arrancar las migraciones/JPA:

Entidad	Campos clave	Relaciones
Usuario	nombre, email, password (hash), rol (ENUM)	1:N con Reserva (como cliente)
Caballo	nombre, identificador, edad, raza, sexo, peso, fotoUrl	1:N con RegistroMedico, PlanAlimentacion, Reserva

Entidad	Campos clave	Relaciones
RegistroMedico	tipo (vacuna/tratamiento/alergia/observación), fecha, descripción, responsableId	N:1 con Caballo, N:1 con Empleado
Empleado	nombre, rol (veterinario/potrador/cuidador/administrador), contacto	1:N con Turno, 1:N con RegistroMedico
Turno	empleadold, fecha, horalnicio, horaFin, tareaAsignada	N:1 con Empleado
Reserva	tipo (cita_vet/monta/paseo/entrenamiento), fecha, horalnicio, horaFin, estado, cupoMaximo, cupoActual, caballold, clienteld	N:1 con Caballo, N:1 con Usuario
PlanAlimentacion	caballold, descripción, frecuencia	1:N con RegistroSuministro
RegistroSuministro	fecha, tipo, cantidad, planId	N:1 con PlanAlimentacion
Insumo	nombre, tipo (alimento/medicina), stockActual, stockMinimo	1:N con MovimientoInsumo (opcional, o solo restar stock directo)
Alerta	tipo, mensaje, referenciald, leida (bool), fecha	N:1 con Usuario (destinatario)

Reserva con cupoMaximo/cupoActual les resuelve de una vez el control de cupo de paseos (3 pts) sin tabla extra.

Estructura de proyecto

Backend (Spring Boot, Maven):

caballeriza-backend/

└─ src/main/java/com/caballeriza/

└─ config/ SecurityConfig, JwtConfig, OpenApiConfig, CorsConfig

└─ controller/ CaballoController, EmpleadoController, ReservaController...

- | └─ dto/ CaballoDTO, request/, response/
- | └─ entity/ Caballo, Empleado, Reserva, RegistroMedico...
- | └─ repository/ interfaces JpaRepository<T, Long>
- | └─ service/ + impl/ (interfaces y lógica de negocio)
- | └─ security/ JwtUtil, JwtFilter, UserDetailsServiceImpl
- | └─ exception/ GlobalExceptionHandler, ResourceNotFoundException
- | └─ CaballerizaApplication.java
- | └─ src/main/resources/
- | └─ application.yml
- | └─ db/migration/ (si usan Flyway: V1__init.sql, V2__seed.sql)
- | └─ src/test/java/... (tests JUnit/Mockito por servicio)
- | └─ Dockerfile
- | └─ pom.xml

Frontend (React + Vite):

caballeriza-frontend/

- | └─ src/
- | └─ api/ axiosClient.js, caballoApi.js, authApi.js...
- | └─ components/ ui/ (Button, Modal, Table), layout/ (Navbar, Sidebar)
- | └─ pages/ Caballos/, Personal/, Calendario/, Alimentacion/, Login/
- | └─ context/ AuthContext.jsx (token, rol, login/logout)
- | └─ routes/ PrivateRoute.jsx, RoleRoute.jsx
- | └─ hooks/ useAuth.js, usePagination.js
- | └─ App.jsx
- | └─ src/tests/ tests básicos con Vitest + Testing Library
- | └─ Dockerfile

└─ package.json

Esta separación por capas (controller/service/repository) es exactamente lo que pide el ítem de "requisitos no funcionales" (4 pts), así que no es trabajo extra, es la base.

Endpoints REST por módulo (mínimo viable)

Módulo	Endpoints clave
Auth	POST /api/auth/registro, POST /api/auth/login (devuelve JWT)
Caballos	GET/POST /api/caballos, GET/PUT/DELETE /api/caballos/{id}, GET/POST /api/caballos/{id}/historial-medico
Personal	GET/POST /api/empleados, PUT/DELETE /api/empleados/{id}, GET/POST /api/empleados/{id}/turnos
Calendario	GET /api/reservas?tipo=&fecha=&page=, POST/PUT/DELETE /api/reservas/{id}, PATCH /api/reservas/{id}/cancelar
Alimentación	GET/POST /api/caballos/{id}/plan-alimentacion, POST /api/planes/{id}/suministros
Inventario	GET/POST /api/insumos, PUT /api/insumos/{id}, GET /api/insumos/stock-bajo
Alertas	GET /api/alertas?leida=false, PATCH /api/alertas/{id}/marcar-leida

Para el ítem de "diseño para móvil" (4 pts), basta con: paginar todos los listados (Pageable de Spring), aceptar filtros como query params (ya están arriba), y devolver errores en un formato JSON consistente ({status, error, message, timestamp}) vía un @ControllerAdvice. Eso es lo que evalúan, no una app móvil real.

División de trabajo (2 personas, por puntaje)

Con 2 personas, lo más eficiente es repartir por **módulo completo** (backend + frontend de ese módulo), no por capas, así evitan choques de Git constantes.

Persona	Módulos (backend + frontend)	Puntos que cubre
A	Seguridad/Auth (lo arranca primero, lo necesitan ambos) + Caballos + Historial médico + Alimentación + Inventario	12 + 15 + 12 = 39

Persona Módulos (backend + frontend)		Puntos que cubre
B	Personal/Turnos + Calendario/Reservas (+ cupo) + Notificaciones/Alertas	12 + 15 + 8 = 35
Juntos	Interfaz general/UX, API docs (Swagger es casi automático con springdoc), tests, README, Docker, video	10 + 12 + 4 = 26

Auth lo hace quien domine mejor Spring Security, porque todo lo demás depende de tener JWT y roles funcionando para poder probar permisos por módulo.

Cronograma sugerido (bloques, no horas exactas)

Bloque	Duración aprox.	Qué hacer
Setup conjunto	2 h	Spring Initializr (web, jpa, security, postgresql, lombok, validation), Vite + React, docker-compose.yml con Postgres, repo en GitHub con ambos como colaboradores, definir DTOs/contrato de API juntos
Auth + esqueleto de entidades	3 h	Persona A: JWT + roles + login/registro funcionando end-to-end. Persona B en paralelo: todas las entidades JPA + repositorios + seed de datos de prueba
Desarrollo en paralelo (vertical)	8-10 h	Cada uno construye su set de módulos completo: controller → service → repo → página React conectada. Probar con Postman/Insomnia antes de conectar frontend
Integración	2-3 h	Unir ambas partes, navbar/menú único, rutas protegidas por rol en frontend, revisar que todos los roles vean lo que deben
Pulido + no funcionales	3 h	Validaciones de formularios, mensajes de error claros, tests mínimos (1-2 por servicio backend, 1-2 de componente en frontend), Swagger revisado
Cierre	2-3 h	README completo (instalación, variables de entorno, cómo correr con Docker), grabar video (≤5 min) mostrando cada punto de la rúbrica, push final, llenar el formulario con URLs

Eso suma ~20-23 horas de trabajo efectivo repartidas entre los dos — apretado pero realista si arrancan ya.

Cosas que NO deben dejar pasar

La foto opcional del caballo: aunque diga "opcional", el rubro de CRUD de caballos (8 pts) menciona explícitamente "upload foto funciona" como criterio, así que sí suma puntos — resuélvanlo simple, un endpoint multipart/form-data que guarde el archivo en disco y almacene la ruta, sin necesidad de un servicio externo como Cloudinary.

El video: máximo 5 minutos, debe recorrer los puntos de la rúbrica y mostrar que la interfaz está pensada para portarse a móvil (no necesitan una app móvil real, solo mencionar/mostrar cómo el diseño responsivo y la API ya lo permiten).

Las notificaciones in-app (3 pts): no necesitan un sistema de scheduler complejo; un endpoint que calcule "alertas" al momento (vacunas próximas, stock bajo) y las guarde/marque como leídas es suficiente para cumplir el criterio.

El control de cupo (3 pts): es fácil de olvidar porque parece un detalle menor dentro de "Calendario", pero tiene su propia línea en la rúbrica — asegúrense de que el backend rechace una reserva de paseo si ya está lleno (HTTP 409, no solo validación visual en el frontend).

README y Docker: "recomendado, no obligatorio" el Docker, pero el README con instrucciones de cómo correr el proyecto en desarrollo sí es parte de los 4 puntos de no funcionales y es rápido de hacer si ya usaron docker-compose.yml desde el principio.